



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Моделирование сред и разработка приложений виртуальной
и дополненной реальности»

Практическая работа № 3

Студент группы *ИКБО-50-23, Павлов Н.С.*

(подпись)

Старший
преподаватель *Горбатов Г.В.*

(подпись)

Отчет представлен «__» _____ 2026 г.

Москва 2026 г.

1. Для проекта был создан интерактивный пользовательский интерфейс. Реализация проводилась с использованием UI(Canvas) и размещением на нем панелей, блоков текста и кнопок. Были созданы, как отдельные сцены, Главное меню и Окно выбора уровня. Полученные окна представлены на рис. 1-2.

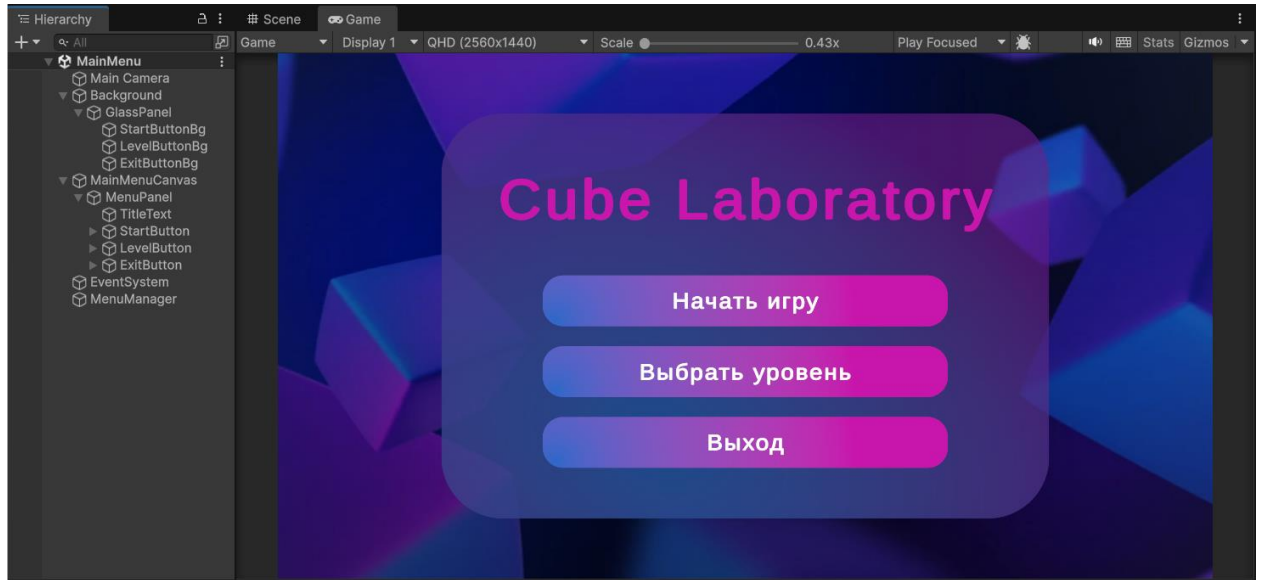


Рисунок 1 — Главное меню

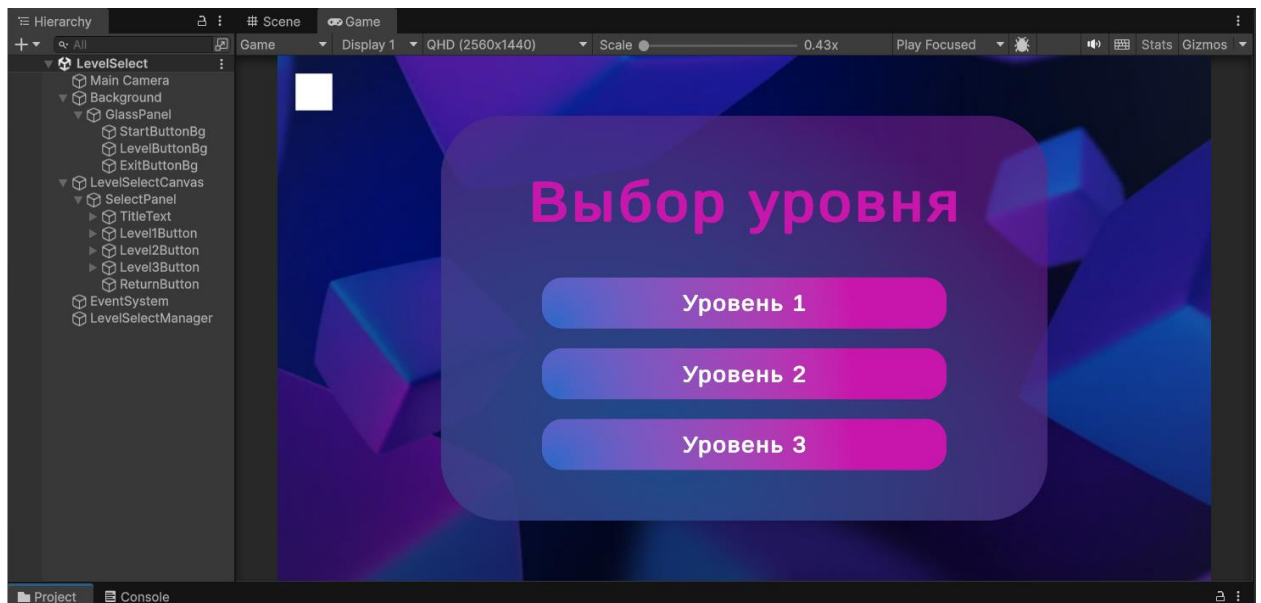


Рисунок 2 — Окно выбора уровня

Для управления логикой кнопок и переходов между экранами используются соответствующие менеджеры (пустые объекты со скриптами)

Листинг 1 - Код скрипта MainMenu.cs на C#

```
using UnityEngine;
using UnityEngine.SceneManagement;

public class MainMenu : MonoBehaviour
{
    public void StartGame()
    {
        GameManager.Instance?.ResetProgress();
        SceneManager.LoadScene("Level1");
    }

    public void OpenLevelSelect()
    {
        SceneManager.LoadScene("LevelSelect");
    }

    public void ExitGame()
    {
        Debug.Log("Выход из игры");

        #if UNITY_EDITOR
            UnityEditor.EditorApplication.isPlaying = false;
        #else
            Application.Quit();
        #endif
    }
}
```

Листинг 2 - Код скрипта LevelSelect.cs на C#

```
using UnityEngine;
using UnityEngine.SceneManagement;

public class LevelSelect : MonoBehaviour
{
    public void LoadLevel1()
    {
        SceneManager.LoadScene("Level1");
    }

    public void LoadLevel2()
    {
        SceneManager.LoadScene("Level2");
    }

    public void LoadLevel3()
    {
        SceneManager.LoadScene("Level3");
    }
}
```

```

public void BackToMenu()
{
    SceneManager.LoadScene("MainMenu");
}

```

Также в рамках уровней реализованы окна Паузы, Победы (а также Победы во всей игре) и Проигрыш.

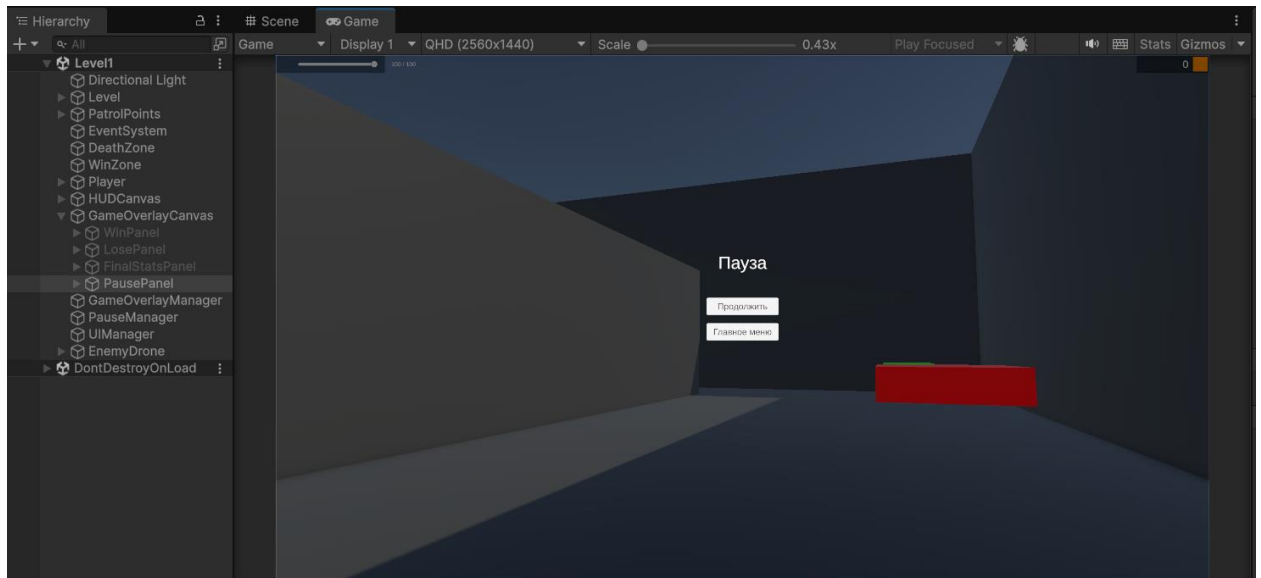


Рисунок 3 — Окно Паузы

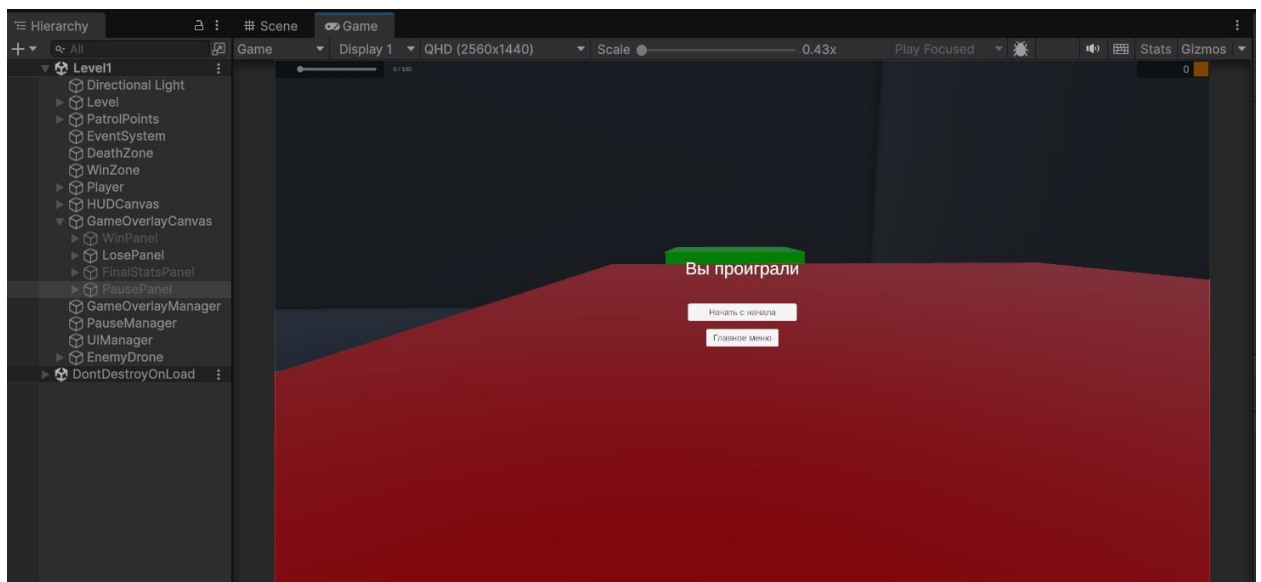


Рисунок 4 — Окно Проигрыша

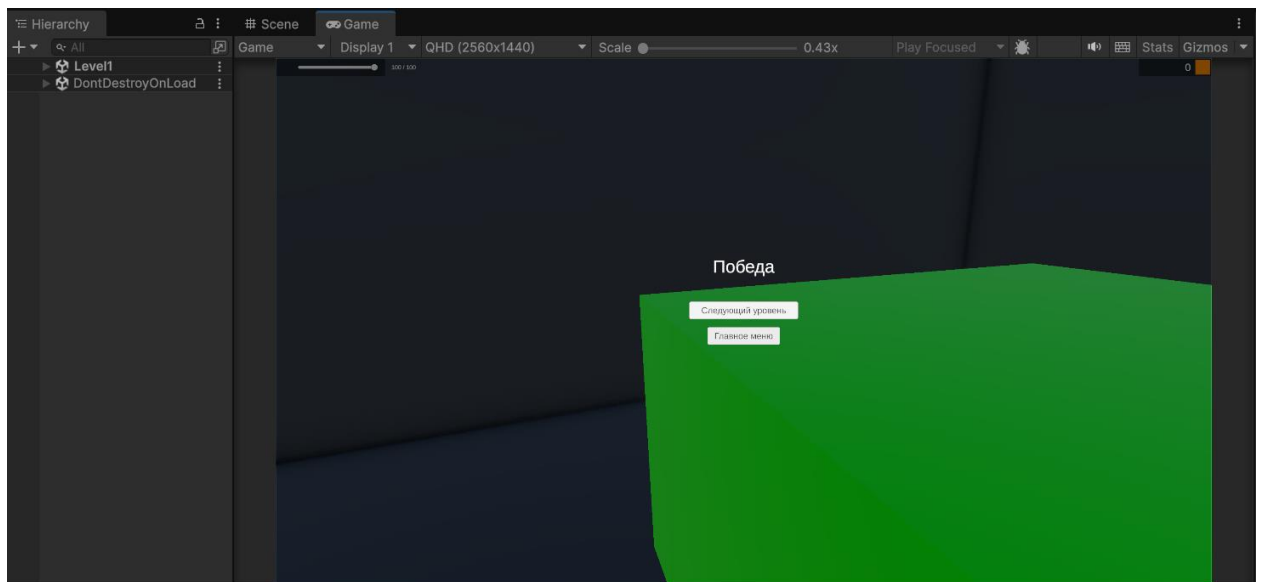


Рисунок 5 — Окно Победы

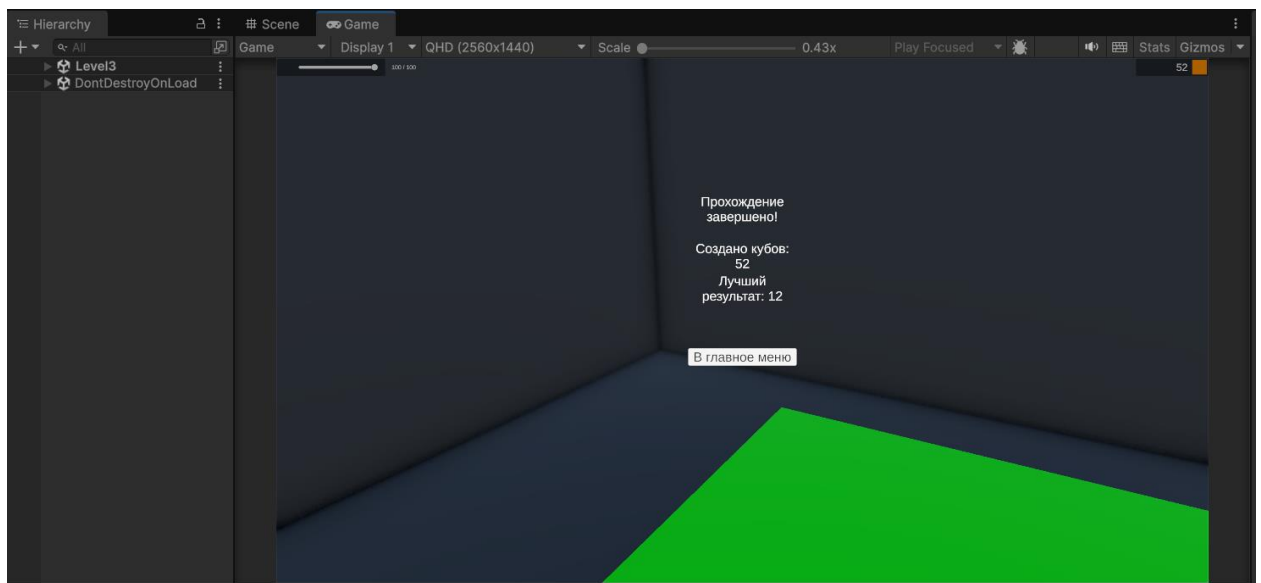


Рисунок 6 — Окно Победы во всей игре

Логика реализуется опять же через соответствующие менеджеры

Листинг 3 - Код скрипта *PauseMenu.cs* на C#

```
using UnityEngine;
using UnityEngine.SceneManagement;

public class PauseMenu : MonoBehaviour
{
    public GameObject pausePanel;
    private bool isPaused = false;

    private void Start()
    {
        pausePanel.SetActive(false);
    }
}
```

```

private void Update()
{
    if (Input.GetKeyDown(KeyCode.Escape))
    {
        if (isPaused)
        {
            ResumeGame();
        }
        else
        {
            PauseGame();
        }
    }
}

public void PauseGame()
{
    isPaused = true;
    pausePanel.SetActive(true);
    Time.timeScale = 0f;
    Cursor.lockState = CursorLockMode.None;
    Cursor.visible = true;
}

public void ResumeGame()
{
    isPaused = false;
    pausePanel.SetActive(false);
    Time.timeScale = 1f;
    Cursor.lockState = CursorLockMode.Locked;
    Cursor.visible = false;
}

public void GoToMainMenu()
{
    Time.timeScale = 1f;
    SceneManager.LoadScene("MainMenu");
}
}

```

Листинг 4 - Код скрипта GameOverlay.cs на C#

```

using UnityEngine;
using UnityEngine.SceneManagement;
using TMPro;

public class GameOverlay : MonoBehaviour
{
    public GameObject winPanel;
    public GameObject losePanel;
    public GameObject finalStatsPanel;
}

```

```

public TextMeshProUGUI cubesUsedText;
public TextMeshProUGUI bestRecordText;
public TextMeshProUGUI newRecordText;

private PlayerController player;
private ObjectGenerator generator;

[System.Obsolete]
private void Start()
{
    player = FindObjectOfType<PlayerController>();
    generator = FindObjectOfType<ObjectGenerator>();

    winPanel.SetActive(false);
    losePanel.SetActive(false);
    if (finalStatsPanel != null) finalStatsPanel.SetActive(false);
}

private void Update()
{
    if (player != null && player.currentHealth <= 0 && !losePanel.activeSelf)
    {
        Lose();
    }
}

public void ShowWin()
{
    winPanel.SetActive(true);
    Time.timeScale = 0f;
    Cursor.lockState = CursorLockMode.None;
    Cursor.visible = true;
}

public void ShowFinalStats(int cubesUsed, int bestScore, bool isNewRecord)
{
    if (finalStatsPanel != null)
    {
        finalStatsPanel.SetActive(true);

        if (cubesUsedText != null)
            cubesUsedText.text = $"Создано кубов: {cubesUsed}";

        if (bestRecordText != null)
            bestRecordText.text = $"Лучший результат: {bestScore}";

        if (newRecordText != null)
        {
            if (isNewRecord)
                newRecordText.text = "НОВЫЙ РЕКОРД! 🏆 ";
        }
    }
}

```

```

        else
            newRecordText.text = "";
    }
}

Time.timeScale = 0f;
Cursor.lockState = CursorLockMode.None;
Cursor.visible = true;
}

public void NextLevel()
{
    GameManager.Instance.LoadNextLevel();
}

public void RestartLevel()
{
    Time.timeScale = 1f;
    SceneManager.LoadScene(SceneManager.GetActiveScene().name);
}

public void GoToMainMenu()
{
    Time.timeScale = 1f;
    SceneManager.LoadScene("MainMenu");
}

private void Lose()
{
    losePanel.SetActive(true);
    Time.timeScale = 0f;
    Cursor.lockState = CursorLockMode.None;
    Cursor.visible = true;
}
}

```

2. Как видно на скриншотах выше, в проект были добавлены показатели, бары (HUD). У пользователя на всех уровнях отображается текущее состояние здоровья и количество использованных кубиков.

Листинг 5 - Код скрипта GameUI.cs на C#

```

using UnityEngine;
using UnityEngine.UI;
using TMPro;

public class GameUI : MonoBehaviour
{
    [Header("Ссылки на UI элементы")]

```



```

public Slider healthSlider;
public TextMeshProUGUI healthText;
public TextMeshProUGUI cubeCountText;

private PlayerController player;

private void Start()
{
    GameObject playerObj = GameObject.FindGameObjectWithTag("Player");
    if (playerObj != null)
    {
        player = playerObj.GetComponent<PlayerController>();
    }

    UpdateHealthUI();
    UpdateCubesUI();
}

private void Update()
{
    UpdateHealthUI();
    UpdateCubesUI();
}

private void UpdateHealthUI()
{
    if (player != null)
    {
        healthSlider.value = player.currentHealth;
        healthText.text = $"{Mathf.RoundToInt(player.currentHealth)} / {Mathf.RoundToInt(player.maxHealth)}";
    }
}

private void UpdateCubesUI()
{
    if (GameManager.Instance != null && cubeCountText != null)
    {
        int totalCubes = GameManager.Instance.GetTotalCubes();
        cubeCountText.text = totalCubes.ToString();
    }
    else if (cubeCountText != null)
    {
        cubeCountText.text = "0";
        Debug.LogWarning("GameManager.Instance не найден в GameUI");
    }
}
}

```

3. В игре реализовано 3 уровня (пример одного из них на рис.7).

Переход между ними, реализован в листингах выше, также как и в свойствах сборки проекта (рис. 8). Для проброса данных между сценами на все уровни добавлен объект GameManager с одноименным скриптом.

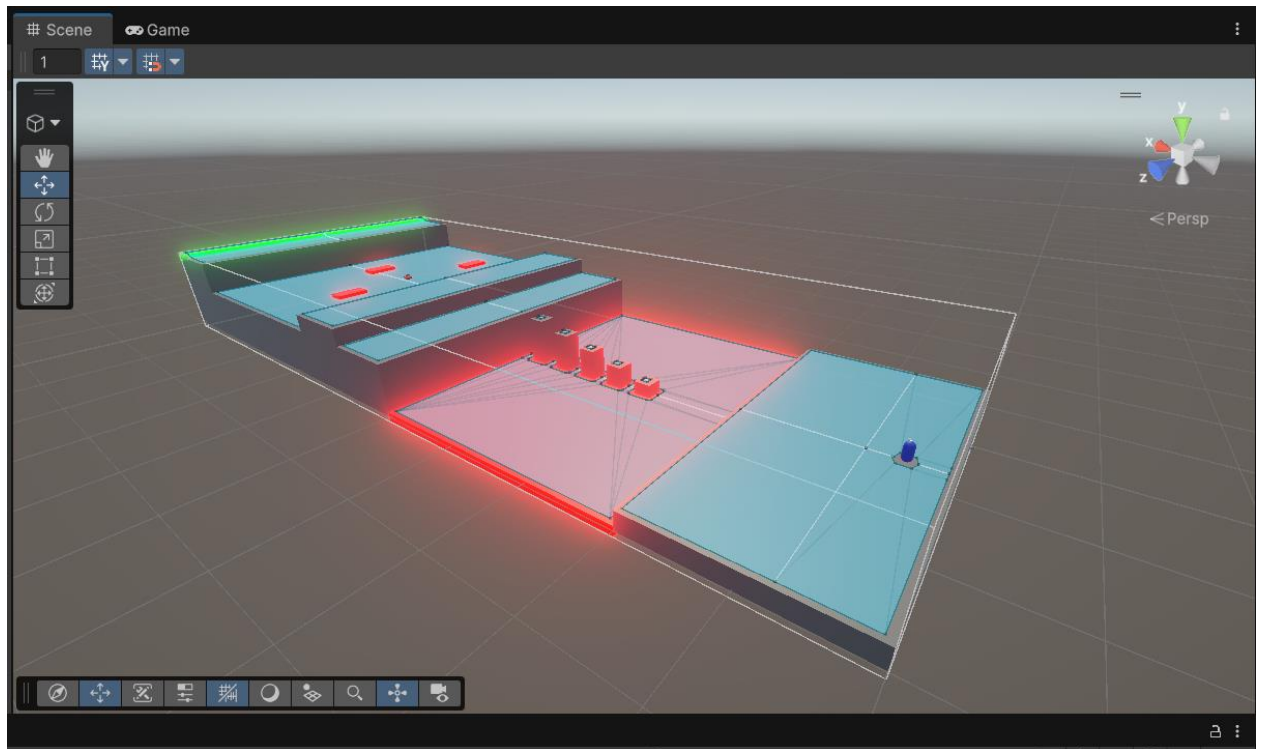


Рисунок 7 — Один из созданных уровней

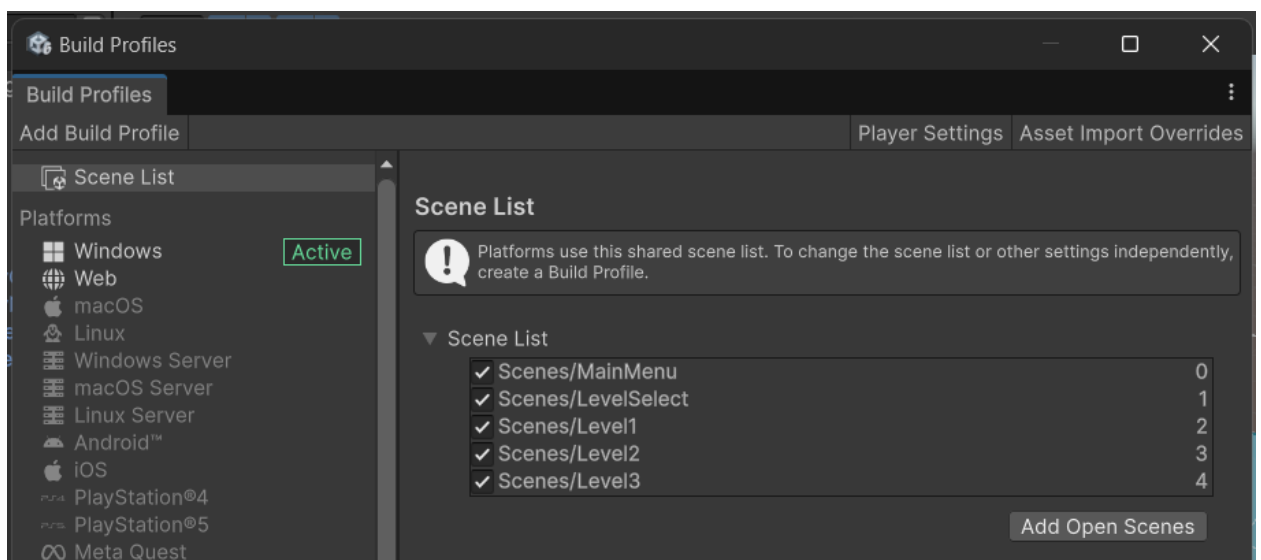


Рисунок 8 — Список сцен проекта

Листинг 6 - Код скрипта GameManager.cs на C#

```
using UnityEngine;  
using UnityEngine.SceneManagement;
```

```

public class GameManager : MonoBehaviour
{
    public static GameManager Instance;

    private int totalCubesCreated = 0;
    private int currentLevelIndex = 1;
    private int bestScore = 99999;

    private bool level1Completed = false;
    private bool level2Completed = false;
    private bool level3Completed = false;

    private void Awake()
    {
        if (Instance == null)
        {
            Instance = this;
            DontDestroyOnLoad(gameObject);
            LoadBestScore();
        }
        else
        {
            Destroy(gameObject);
        }
    }

    private void Start()
    {
        SceneManager.sceneLoaded += OnSceneLoaded;
    }

    private void OnSceneLoaded(Scene scene, LoadSceneMode mode)
    {
        string sceneName = scene.name;

        if (sceneName == "Level1") currentLevelIndex = 1;
        else if (sceneName == "Level2") currentLevelIndex = 2;
        else if (sceneName == "Level3") currentLevelIndex = 3;

        Debug.Log($"Загружен уровень {currentLevelIndex}. Всего создано кубов: {totalCubesCreated}");
    }

    public void AddCube()
    {
        totalCubesCreated++;
        Debug.Log($"Создан куб! Всего: {totalCubesCreated}");
    }

    public int GetTotalCubes()
    {

```

```

        return totalCubesCreated;
    }

[System.Obsolete]
public void CompleteLevel()
{
    switch (currentLevelIndex)
    {
        case 1:
            level1Completed = true;
            break;
        case 2:
            level2Completed = true;
            break;
        case 3:
            level3Completed = true;
            break;
    }

    if (currentLevelIndex == 3)
    {
        ShowFinalStats();
    }
    else
    {
        ShowWinPanel();
    }
}

[System.Obsolete]
private void ShowWinPanel()
{
    GameOverlay overlay = FindObjectOfType<GameOverlay>();
    if (overlay != null)
    {
        overlay.ShowWin();
    }
}

[System.Obsolete]
private void ShowFinalStats()
{
    bool isNewRecord = totalCubesCreated < bestScore;

    if (isNewRecord)
    {
        bestScore = totalCubesCreated;
        SaveBestScore();
    }

    GameOverlay overlay = FindObjectOfType<GameOverlay>();

```

```
        if (overlay != null)
        {
            overlay.ShowFinalStats(totalCubesCreated, bestScore, isNewRecord);
        }
    }

    private void SaveBestScore()
    {
        PlayerPrefs.SetInt("BestScore", bestScore);
        PlayerPrefs.Save();
        Debug.Log($"Новый рекорд сохранен: {bestScore}");
    }

    private void LoadBestScore()
    {
        if (PlayerPrefs.HasKey("BestScore"))
        {
            bestScore = PlayerPrefs.GetInt("BestScore");
            Debug.Log($"Загружен рекорд: {bestScore}");
        }
        else
        {
            bestScore = 99999;
            Debug.Log("Рекорда еще нет");
        }
    }

    public void ResetProgress()
    {
        totalCubesCreated = 0;
        level1Completed = false;
        level2Completed = false;
        level3Completed = false;
        currentLevelIndex = 1;
        Debug.Log("Прогресс сброшен");
    }

    public void LoadNextLevel()
    {
        string nextLevel = "";

        switch (currentLevelIndex)
        {
            case 1:
                nextLevel = "Level2";
                break;
            case 2:
                nextLevel = "Level3";
                break;
            case 3:
                nextLevel = "MainMenu";
                break;
        }
    }
}
```

```
        break;
    }

    Time.timeScale = 1f;
    SceneManager.LoadScene(nextLevel);
}

private void OnDestroy()
{
    SceneManager.sceneLoaded -= OnSceneLoaded;
}
}
```